시스템 프로그래밍

9장. 프로세스 제어

컴퓨터소프트웨어학부

School of Computer Software at Daegu Catholic University © 2021

목차

- 실습 문제 (13문제)
  - ✓ 1. 프로세스 생성 (3)
  - ✓ 2. 프로세스 기다리기 (2)
  - ✓ 3. 프로그램 실행 (3)
  - ✓ 4. system() 함수 구현 (1)
  - ✓ 5. 입출력 재지정 (2)
  - ✓ 6. 프로세스 그룹 (2)
- 과제 (4문제)
  - ✓ 1. 전면처리, 후면처리 실습
  - ✓ 2. 작업제어 실습
  - ✓ 3. 쉘 인터프리터 작성_1
  - ✓ 4. 쉘 인터프리터 작성_2

실습 문제 1-1 – 프로세스 생성

- 자식 프로세스 생성 후 getpid() 함수를 이용해 프로세스 return 값을 확인 하시오.

✓참조 자료 : 강의자료 6번 슬라이드

1. printf()에서 getpid() 함수를 이용해 부모 프로세스 id를 출력 하시오.
2. fork() 함수를 이용해 변수 'pid'에 자식 프로세스 id 값을 할당 하시오.
3. printf()에서 getpid() 함수를 이용해 자식 프로세스 id와 자식 프로세스 return 값을 출력 하시오.

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9
[27323] Process start
[27323] Process : return value 27324
[27324] Process : return value 0

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 1-2 – 프로세스 생성

- 자식 프로세스를 생성하고 부모 프로세스와 각기 다른 메시지를 출력 하시오.

✓참조 자료 : 강의자료 8번 슬라이드

1. 변수 'pid'에 fork() 함수를 이용해 자식 프로세스 return 값을 할당 받으시오.
2. If() 문을 통해 자식 프로세스의 return 값이 0이면 아래와 같은 문장을 출력 하시오.
   - "[Child] Hello, world! Pid=%d\n"
3. 그외에 프로세스는 부모 프로세스로 아래와 같은 문장을 출력 하시오.
   - "[Parent] Hello, world! Pid=%d\n"

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9
[parent] : Hello, World pid = 27557
[child] : Hello, World pid = 27558

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 1-3 – 프로세스 생성

- 두 개의 자식 프로세스를 생성 하시오.

  ✓참조 자료 : 강의자료 9번 슬라이드

  1. 변수 ‘pid’에 fork() 함수를 이용해 자식 프로세스 return 값을 할당 받으시오.
  2. If() 문을 이용하여 pid1과 pid2 return 값이 0일 때 각각 프로세스 번호를 출력 하시오.
     - 단, print() 출력문에서 출력할 값은 프로세스 번호이다.

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9
[child 2] : Hello, world ! pid = 27764
[child 1] : Hello, world ! pid = 27763

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 2-1 – 프로세스 기다리기

- wait() 함수를 이용해 자식 프로세스가 끝나기를 기다리는 프로그램을 작성 하시오.

✓참조 자료 : 강의자료 11번 슬라이드

1. 변수 'pid'에 fork() 함수를 이용해 자식 프로세스 return 값을 할당 받으시오.
2. If() 문을 통해 자식 프로세스의 return 값이 0이면 아래와 같은 문장을 출력 하시오.
   - "[Child] Hello, world! Pid=%d\n"
   - 단, exit(1)을 이용해 자식 프로세스를 끝내시오.
3. wait() 함수를 이용해 부모 프로세스를 대기 상태로 만들고 자식 프로세스를 끝내시오.
   - Wait() 함수 동작이 성공 시 자식 프로세스를 return 값으로 받으며, 실패 시 -1을 반환한다.
   - 따라서, 반환된 값을 출력하는 출력문을 작성 하시오.
4. status 상태 또한 출력문으로 출력 하시오.

실습 문제 2-1 – 프로세스 기다리기

- wait() 함수를 이용해 자식 프로세스가 끝나기를 기다리는 프로그램을 작성 하시오.
  - ✓참조 자료 : 강의자료 11번 슬라이드

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9
[29214] parental process start
[29215] child process start
[29214] child process end 29215
     end code 1

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 2-2 – 프로세스 기다리기

- wait() 함수를 이용해 자식 프로세스가 끝나기를 기다리는 프로그램을 작성 하시오.

  ✓참조 자료 : 강의자료 12, 13번 슬라이드

  1. 변수 ‘pid1’에 fork() 함수를 이용해 자식 프로세스 return 값을 할당 받으시오.
  2. If() 문을 통해 자식 프로세스의 return 값이 0이면 아래와 같은 조건을 출력 하시오.
     - 출력문 “[프로세스 ID] child process start” 출력
     - 함수 sleep()을 이용해 1ms에 해당하는 딜레이를 주고 출력문 “[프로세스 ID] child process end” 을 출력 하시오.
     - 단, exit(1)을 이용해 자식 프로세스를 끝내시오.
  3. 수행 항목 1, 2를 한 번 더 수행하되 변수는 ‘pid2’로 수행하며, sleep()과 exit()의 값은 2로 수정 하시오.

실습 문제 2-2 – 프로세스 기다리기

- wait() 함수를 이용해 자식 프로세스가 끝나기를 기다리는 프로그램을 작성 하시오.

✓참조 자료 : 강의자료 12, 13번 슬라이드

4. wait() 함수를 이용해 부모 프로세스를 대기 상태로 만들고 자식 프로세스를 끝내시오.
   - Wait() 함수 동작이 성공 시 자식 프로세스를 return 값으로 받으며, 실패 시 -1을 반환한다.
   - 따라서, 반환된 값을 출력하는 출력문을 작성 하시오.
5. status 상태 또한 출력문으로 출력 하시오.

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9
[29778] parental process start
[29779] child process[1] start
[29780] child process[2] start
[29779] child process[1] end
[29778] child process[1] 29779 end
     end code 1

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 3-1 – 프로그램 실행

- fork() 함수로 자식 프로세스를 생성하고, execl()함수를 이용해 echo 명령어를 실행 하시오.

  ✓참조 자료 : 강의 자료 21번 슬라이드

  1. 부모 프로세스의 시작을 알리는 출력문을 작성 하시오.
  2. 자식 프로세스를 생성함과 동시에 if() 문을 통해 return 값을 확인 하시오.
     - execl() 함수를 이용해 echo 명령어를 실행 하시오.
     - echo 명령어 파일은 “/bin/echo“이며, 출력할 문구는 “hello”이다.
     - execl() 함수가 잘못되어 출력을 못할 경우 fprintf()를 이용해 stderr를 출력 하시오.
  3. 부모 프로세스의 종료를 알리는 출력문을 작성 하시오.

실습 문제 3-1 – 프로그램 실행

- fork() 함수로 자식 프로세스를 생성하고, execl()함수를 이용해 echo 명령어를 실행 하시오.
  - ✓참조 자료 : 강의 자료 21번 슬라이드

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9
parental process start
parental process end
hello

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 3-2 – 프로그램 실행

- 세 개의 자식 프로세스를 생성하여 각각 다른 명령어를 실행 하시오.

✓참조 자료 : 강의 자료 22번 슬라이드

1. 부모 프로세스의 시작을 알리는 출력문을 작성 하시오.
2. 자식 프로세스를 생성함과 동시에 if() 문을 통해 return 값을 확인 하시오.
   - execl() 함수를 이용해 echo 명령어를 실행 하시오.
   - echo 명령어 파일은 “/bin/echo“이며, 출력할 문구는 “hello”이다.
   - execl() 함수가 잘못되어 출력을 못할 경우 fprintf()를 이용해 stderr를 출력 하시오.
3. 자식 프로세스를 생성함과 동시에 if() 문을 통해 return 값을 확인 하시오.
   - execl() 함수를 이용해 date 명령어를 실행 하시오.
   - date 명령어 파일은 “/bin/date“이며, 추가 옵션은 없다.
   - execl() 함수가 잘못되어 출력을 못할 경우 fprintf()를 이용해 stderr를 출력 하시오.

실습 문제 3-2 – 프로그램 실행

- 세 개의 자식 프로세스를 생성하여 각각 다른 명령어를 실행 하시오.

✓참조 자료 : 강의 자료 22번 슬라이드

4. 자식 프로세스를 생성함과 동시에 if() 문을 통해 return 값을 확인 하시오.
   - execl() 함수를 이용해 ls 명령어를 실행 하시오.
   - ls 명령어 파일은 “/bin/ls“이며, 추가 옵션은 –l 이다.
   - execl() 함수가 잘못되어 출력을 못할 경우 fprintf()를 이용해 stderr를 출력 하시오.
5. 부모 프로세스의 종료를 알리는 출력문을 작성 하시오.

실습 문제 3-2 – 프로그램 실행

- 세 개의 자식 프로세스를 생성하여 각각 다른 명령어를 실행 하시오.
  - ✓참조 자료 : 강의 자료 22번 슬라이드

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9
parental process start
parental process end
hello
Thu 30 Sep 2021 05:50:16 AM PDT
total 96
-rw-rw-r-- 1 snslab snslab 46018 Sep 30 01:23 CMakeCache.txt
drwxrwxr-x 5 snslab snslab  4096 Sep 30 05:50 CMakeFiles
-rw-rw-r-- 1 snslab snslab  1657 Sep 30 01:22 cmake_install.cmake
-rw-rw-r-- 1 snslab snslab  5329 Sep 30 01:30 Makefile
drwxrwxr-x 3 snslab snslab  4096 Sep 30 01:22 Testing
-rwxrwxr-x 1 snslab snslab 19688 Sep 30 05:49 week9
-rw-rw-r-- 1 snslab snslab  5366 Sep 30 01:30 week9.cbp

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 3-3 – 프로그램 실행

- 명령줄 인자(argument)를 입력 받아 execvp() 함수를 이용해 명령어를 실행 하시오.

  ✓참조 자료 : 강의 자료 23번 슬라이드

  1. 변수 ‘pid1’에 fork() 함수를 이용해 자식 프로세스 return 값을 할당 받으시오.
  2. If() 문을 통해 자식 프로세스의 return 값이 0이면 아래와 같은 조건을 출력 하시오.
     - execvp() 함수에 입력 받은 인자 값을 파라미터로 사용하여 명령어를 실행 하시오.
     - 인자 값이 잘못 되어 execvp() 함수를 실행하지 못할 경우 fprintf()를 이용해 stderr를 출력 하시오.
  3. 자식 프로세스를 생성하지 못했을 경우에 대해 작성 하시오.
     - Wait() 함수를 이용해 자식 프로세스가 끝날 때까지 기다리시오.
     - 자식 프로세스가 끝나면 프로세스 ID와 같이 종료 출력문을 작성 하시오.

실습 문제 3-3 – 프로그램 실행

- 명령줄 인자(argument)를 입력 받아 execvp() 함수를 이용해 명령어를 실행 하시오.
  - ✓참조 자료 : 강의 자료 23번 슬라이드

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9 wc test.txt
 3  7 40 test.txt
[31607] child process 31608 end
     end code 0

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 4 – system() 함수 구현

- system() 함수를 작성 후 main() 함수에서 세 가지 명령어를 입력 받아 실행 하시오.

✓참조 자료 : 강의 자료 25,26번 슬라이드

1. system() 함수를 구현 하시오.
   - 강의 자료 25번 슬라이드 내용을 참조하여 작성하시오.
2. main()에서 system() 함수를 이용해 아래와 같은 조건을 수행 하시오.
   - date 명령어를 파라미터로 사용하며, 변수 'status'에 system() 함수의 반환 값을 할당 하시오.
     - 입력 파라미터의 반환이 잘못 되었을 경우 perror() 함수를 통해 오류 문구를 출력 하시오.
     - date 명령어는 정상 출력이 되어야 하며, 반환 코드는 정상 동작인 0이어야 한다.

실습 문제 4 – system() 함수 구현

- system() 함수를 작성 후 main() 함수에서 세 가지 명령어를 입력 받아 실행 하시오.

✓참조 자료 : 강의 자료 25,26번 슬라이드

- hello 명령어를 파라미터로 사용하며, 변수 'status'에 system() 함수의 반환 값을 할당 하시오.
  - 입력 파라미터의 반환이 잘못 되었을 경우 perror() 함수를 통해 오류 문구를 출력 하시오.
  - hello 명령어는 정상 출력이 되지 않아야 하며, 반환 코드는 정상 동작이 아니기 때문에 127이어야 한다.
- who 명령어와 exit 44를 파라미터로 사용하며, 변수 'status'에 system() 함수의 반환 값을 할당 하시오.
  - 입력 파라미터의 반환이 잘못 되었을 경우 perror() 함수를 통해 오류 문구를 출력 하시오.
  - who 명령어는 정상 출력이 되어야 하며, 반환 코드는 입력 파라미터에서 설정한 것과 같이 44이어야 한다.

실습 문제 4 – system() 함수 구현

- system() 함수를 작성 후 main() 함수에서 세 가지 명령어를 입력 받아 실행 하시오.
  - ✓참조 자료 : 강의 자료 25,26번 슬라이드

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9
Thu 30 Sep 2021 06:14:32 AM PDT
end code 0
end code 127
snslab   :0           2021-09-30 00:58 (:0)
end code 44
sh: 1: hello: not found

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 5-1 – 입출력 재지정

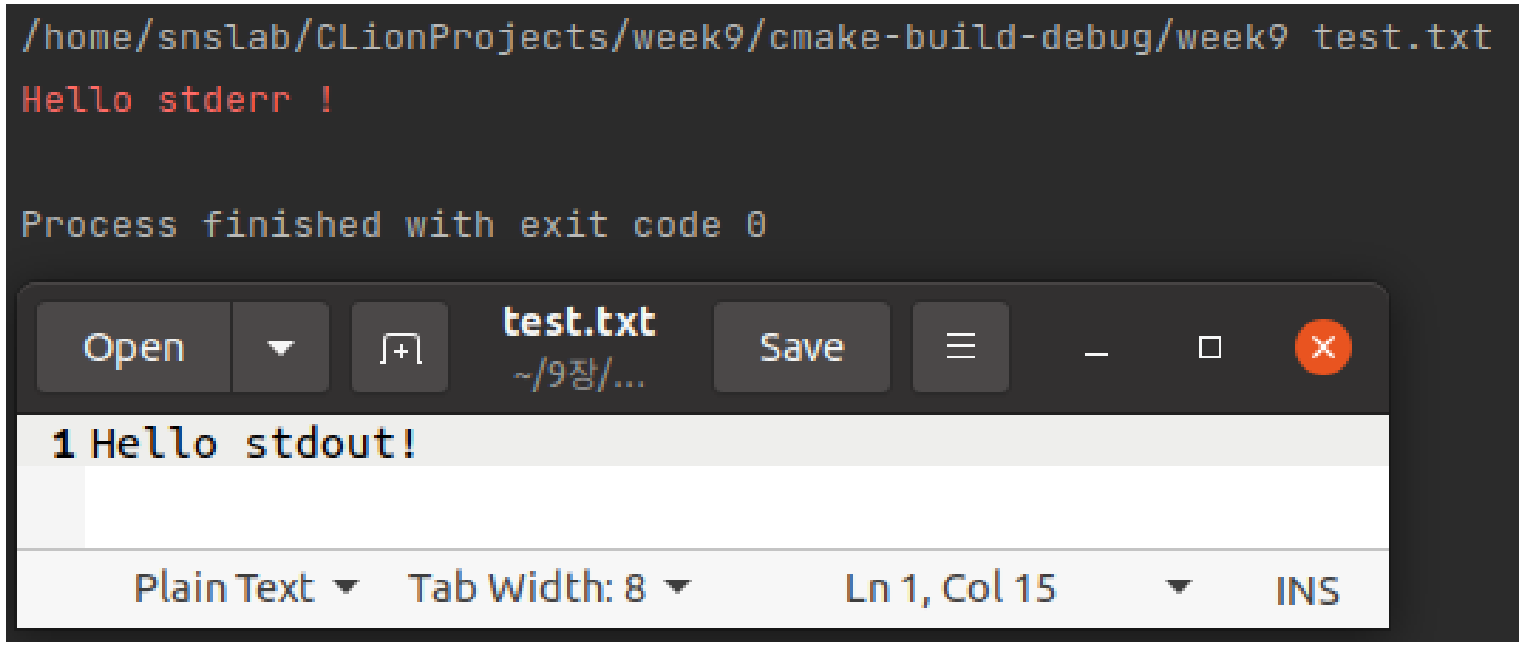
- dup2() 함수를 이용하여 표준 출력을 파일로 지정 하시오.

  ✓참조 자료 : 강의 자료 29번 슬라이드

  1. 입력 인자(argument[1])에 해당하는 파일 디스크립터를 O_CREAT, O_TRUNC, O_WRONLY, 0660 옵션으로 생성 하시오.
  2. dup2() 함수를 이용해 표준 출력을 변경 하시오.
  3. close() 함수를 이용해 파일 디스크립터를 닫으시오.
  4. printf() 문을 이용해 출력 되는 구문의 출력 위치를 확인 하시오.
     - 출력문 “Hello stdout” 출력 -> 입력 인자로 받은 파일에 출력
  5. fprintf()를 이용해 파일 출력을 시도 후 출력 위치를 확인 하시오.
     - 출력문 “Hello stderr !” 출력 -> 터미널 창 출력

실습 문제 5-1 – 입출력 재지정

- dup2() 함수를 이용하여 표준 출력을 파일로 지정 하시오.
  - ✓참조 자료 : 강의 자료 29번 슬라이드



< 결과 화면 >

실습 문제 5-2 – 입출력 재지정

- 입력 인자로 파일명, 명령어, 파일을 입력 받아 dup2() 함수를 이용해 출력 하시오.

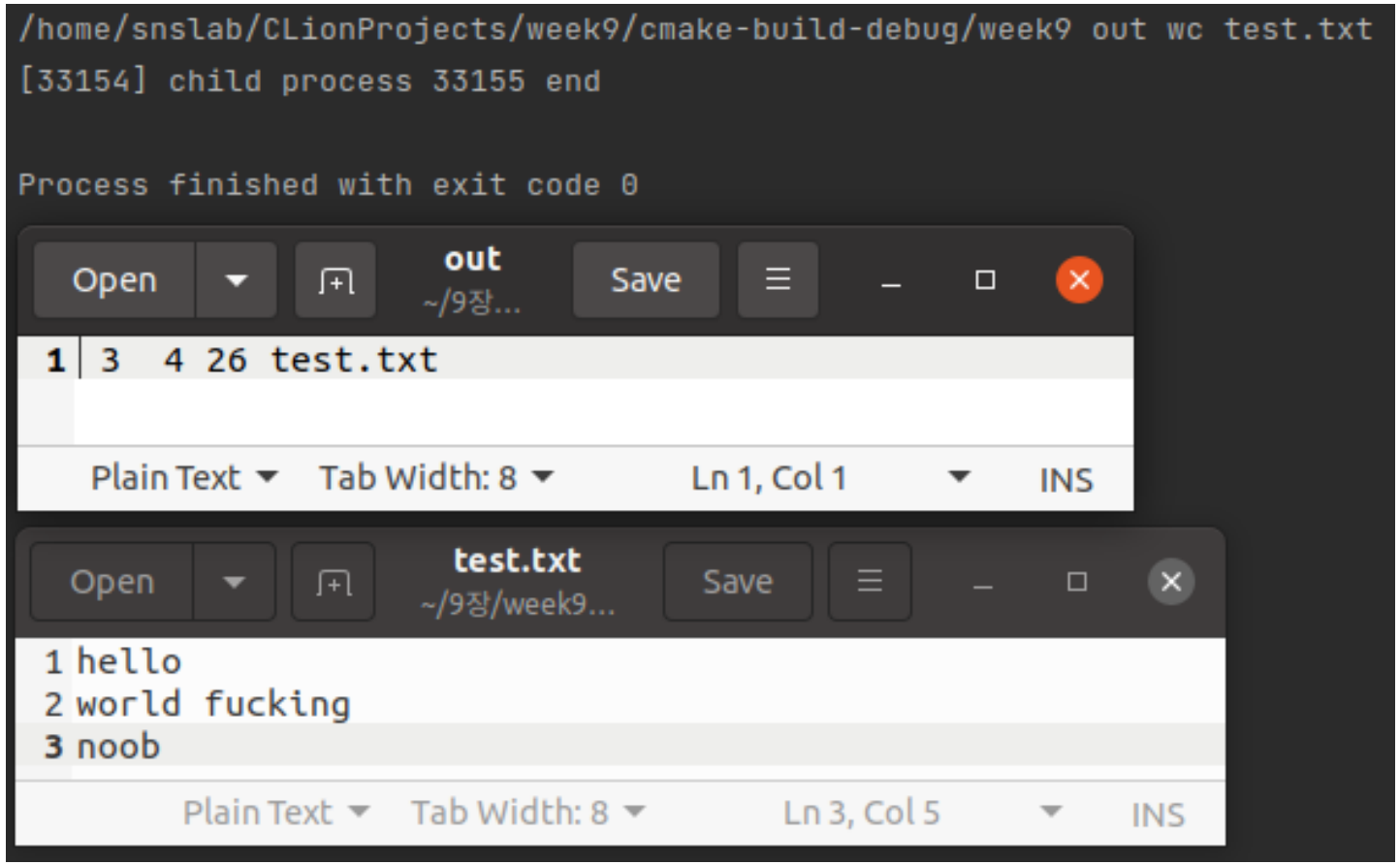
  ✓참조 자료 : 강의 자료 30번 슬라이드

  1. 자식 프로세스 생성 후 if() 문을 통해 return 값을 확인 하시오.
     - 입력 인자(argument[1])에 해당하는 파일 디스크립터를 O_CREAT, O_TRUNC, O_WRONLY, 0660 옵션으로 생성 하시오.
     - dup2() 함수를 이용해 표준 출력을 변경 하시오.
     - close() 함수를 이용해 파일 디스크립터를 닫으시오.
     - execvp() 함수를 이용해 입력 인자로 받은 명령어를 수행 하시오.
  2. 자식 프로세스가 생성되지 않으면, 재생성 될 때까지 wait() 함수를 이용해 대기한다.
     - 변수 status 상태에 따라 자식 프로세스가 종료 되었으면 프로세스 ID와 함께 자식 프로세스의 종료를 printf() 문으로 출력한다.

실습 문제 5-2 – 입출력 재지정

- 입력 인자로 파일명, 명령어, 파일을 입력 받아 dup2() 함수를 이용해 출력 하시오.

✓참조 자료 : 강의 자료 30번 슬라이드



< 결과 화면 >

실습 문제 6-1 – 프로세스 그룹

- 자식 프로세스를 생성하고, getpgrp() 함수를 이용해 프로세스를 그룹화 하시오.

  ✓차조 자료 : 강의 자료 34번 슬라이드

  1. printf()에서 getpid() 함수와 getpgrp() 함수를 이용해 부모 프로세스 ID와 그룹 ID를 출력 하시오.
  2. fork() 함수를 이용해 변수 'pid'에 자식 프로세스 id 값을 할당 하시오.
  3. if() 문을 통해 자식 프로세스의 return 값이 0이면 자식 프로세스의 ID와 그룹 ID를 출력 하시오.

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9 out wc test.txt
[PARENT] PID = 33603 GID = 33603
CHILD : PID = 33604 GID = 33603

Process finished with exit code 0
```

< 결과 화면 >

실습 문제 6-2 – 프로세스 그룹

- 자식 프로세스를 생성하고, setpgid() 함수를 이용해 프로세스를 그룹화 하시오.

  ✓차조 자료 : 강의 자료 37번 슬라이드

  1. printf()에서 getpid() 함수와 getpgrp() 함수를 이용해 부모 프로세스 ID와 그룹 ID를 출력 하시오.
  2. fork() 함수를 이용해 변수 'pid'에 자식 프로세스 id 값을 할당 하시오.
  3. if() 문을 통해 자식 프로세스의 return 값이 0이면 자식 프로세스의 ID와 그룹 ID를 출력 하시오.
     - 단, setpgid() 함수를 이용해 새로운 프로세스 그룹을 생성하고, 자식 프로세스와 부모 프로세스가 다른 그룹의 멤버인지 확인 하시오.

```
/home/snslab/CLionProjects/week9/cmake-build-debug/week9 out wc test.txt
PARENT : PID = 33760 GID = 33760
CHILD : PID = 33761 GID = 33761

Process finished with exit code 0
```

< 결과 화면 >

1.7 과제1 – 전면처리, 후면처리 실습

1. 다음 명령을 실행하고 출력 결과에 대해 설명 하시오.
   - sleep 100 &
   - sleep 100 &
   - jobs
   - ps –f
   - kill %1
   - ps -f



과제2 – 작업제어 실습

2. 다음 명령을 실행하고 출력 결과에 대해 설명 하시오.
   - (echo START1; sleep 10; echo DONE1) &
   - (echo START2; sleep 10; echo DONE2) &
   - kill –KILL 첫 번째 프로세스
   - kill –STOP 첫 번째 프로세스
   - kill –CONT 첫 번째 프로세스



과제3 – 쉘 인터프리터 작성1

3. 다음과 같은 기능을 포함하는 쉘 인터프리터를 작성 하시오.
   - 명령어 실행
     - [shell] cmd
   - 명령어 순차적 실행
     - [shell] cmd1; cmd2; cmd3
   - 후면 실행
     - [shell] cmd &
   - 입출력 리디렉션
     - [shell] cmd > outfile
     - [shell] cmd < infile



1.7 과제4 – 쉘 인터프리터 작성2

4. 프로그램 exec2.c를 참고하여 다음과 같은 기능을 포함하는 쉘 인터프리터를 작성 하시오.
   프로그램 exec2.c는 무조건 echo 명령어를 실행하고 있다. 여기에서는 실행할 명령어를 다음과 같은 형태로 입력 받는다.
   1) 명령어 전면 실행 : [Shell] 명령어
   2) 명령어 후면 실행 : [Shell] 명령어 &

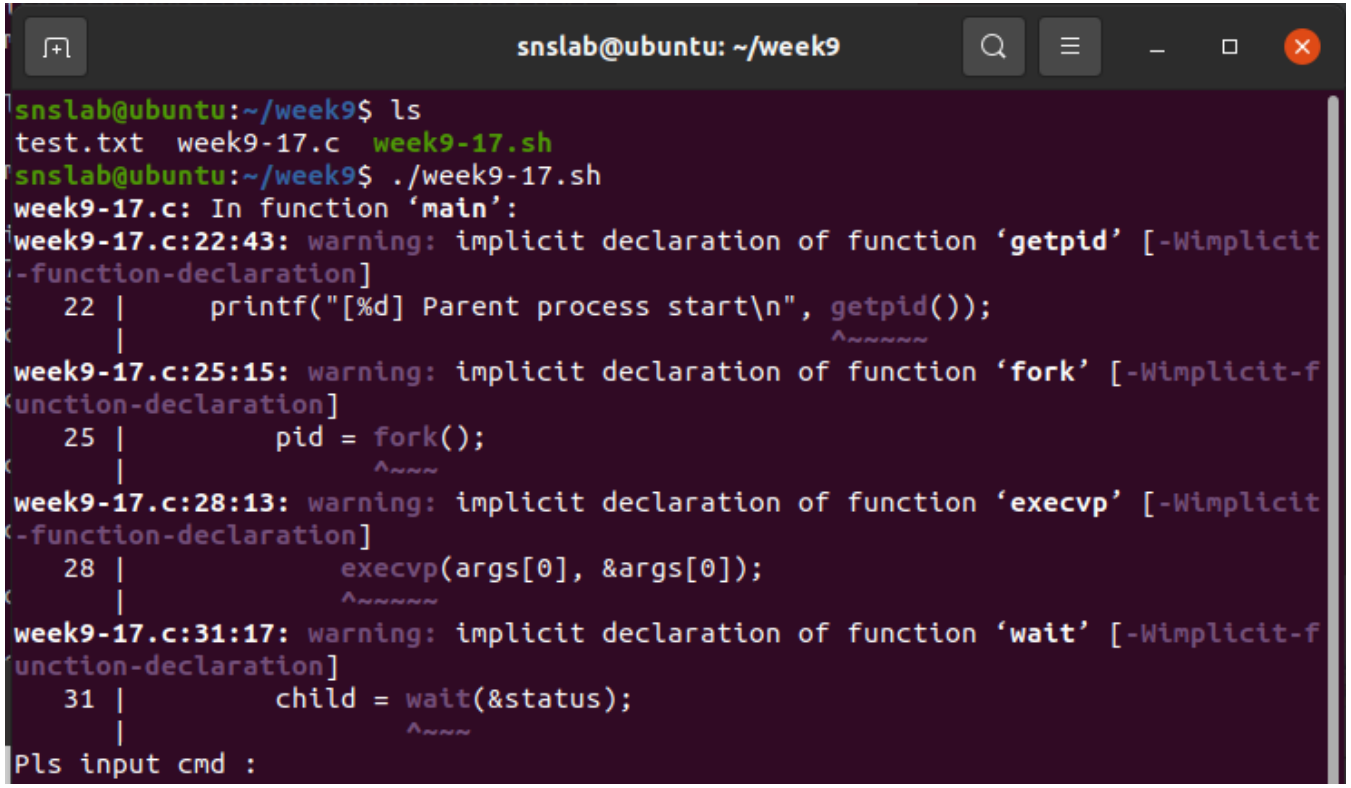
➢ 이 인터프리터를 구현하기 위해서는 다음과 같이 진행한다.

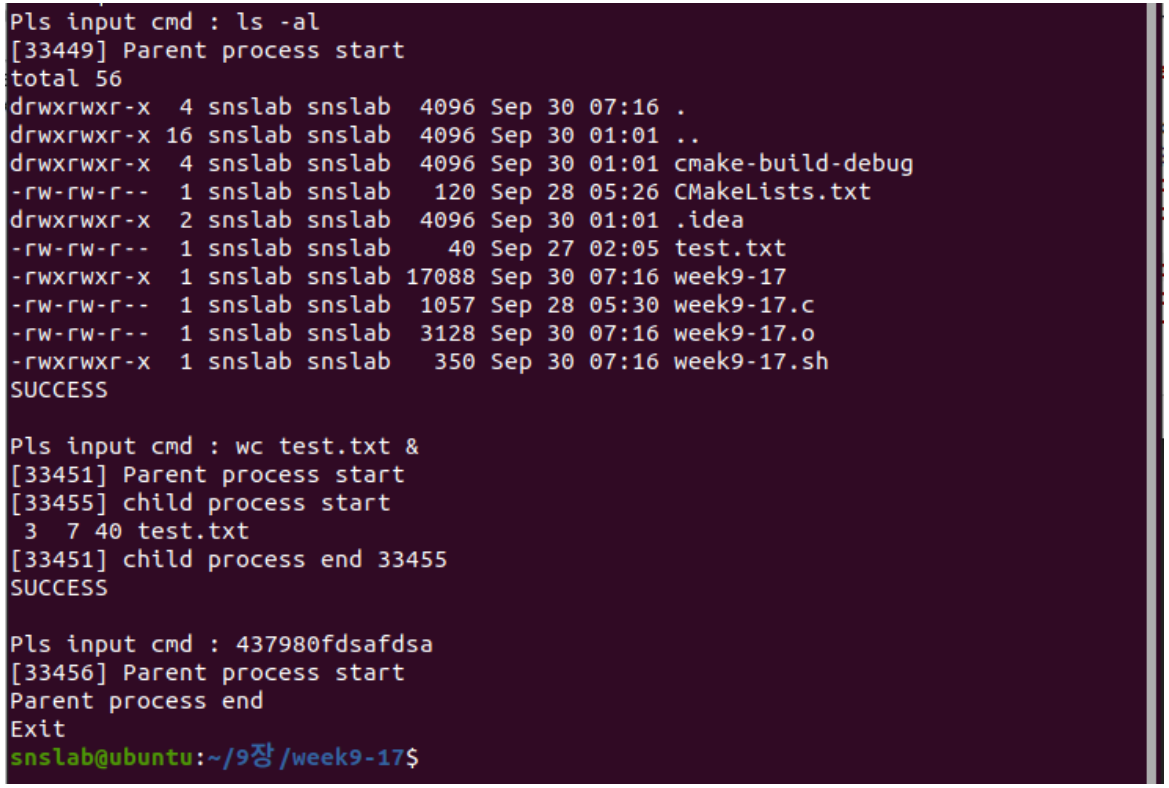
1. 프롬프트를 내주고 명령어를 입력으로 받는다.
2. strtok_r() 함수를 이용하여 이를 명령줄 인수로 분리한다. 분리된 명령줄 인수로 분리한다. 분리된 명령줄 인수를 다음 배열에 순차적으로 대입한다.
   Char *args[MAXARG];
3. 자식 프로세스를 생성하여 자식 프로세스로 하여금 execvp() 시스템 호출을 이용하여 명령어를 실행하게 한다.
4. 부모 프로세스는 자식 프로세스가 끝나기를 기다린다.
5. (1)로 돌아가서 같은 과정을 반복한다.



1.7 과제4 – 쉘 인터프리터 작성2

4. 프로그램 exec2.c를 참고하여 다음과 같은 기능을 포함하는 쉘 인터프리터를 작성 하시오.
   프로그램 exec2.c는 무조건 echo 명령어를 실행하고 있다. 여기에서는 실행할 명령어를 다음과 같은 형태로 입력 받는다.





E N D





School of Computer Software at Daegu Catholic University © 2021